

Mensch-Maschine-Interaktion (WS 24/25)

Übungsblatt 4

Für dieses Übungsblatt muss die Abgabe spätestens bis zum 14.11.24 um 12 Uhr erfolgen. Die Abgaben sind elektronisch an die E-Mail-Adresse mmiuebung@ls7.cs.tu-dortmund.de zu senden. Denken Sie bitte an die korrekte Betreffsformatierung

„MMI GruppeX BlattYY Vorname1 Nachname1 Vorname2 Nachname2 ...“

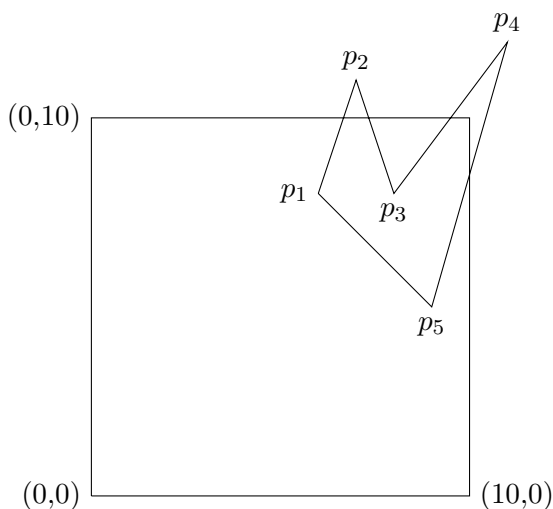
und fügen Sie Ihren Abgaben alle Namen und die Gruppennummer hinzu. Bitte beachten Sie die Hinweise zu den Übungen auf der Webseite <https://moodle.tu-dortmund.de/course/view.php?id=48995>.

Aufgabe 4.1 (Clipping)

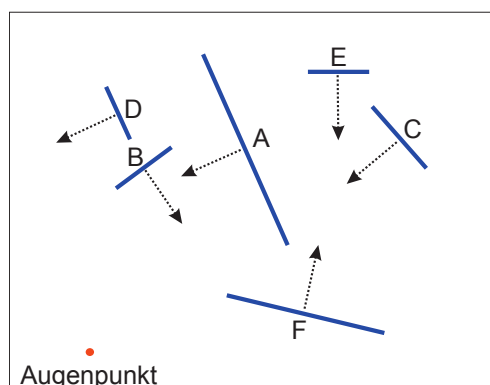
[3 Punkte]

In Abbildung 1(a) ist ein Polygon dargestellt, das auf den Bereich des Rechtecks $((0,0),(0,10),(10,10),(10,0))$ geclippt werden soll.

- Skizzieren Sie jeweils das Ergebnis nach dem Clipping des Polygons bezüglich der Halbebene, die von der oberen Kante induziert wird, und anschließend bezüglich der Halbebene, die von der rechten Kante induziert wird, siehe Algorithmus im Skript S.316 (Kapitel D.4.2.).
- Berechnen Sie die Koordinaten des neuen Eckpunkts p_{12} des Polygons, der nach dem Clipping der Kante $\overline{p_1 p_2}$ entsteht. Die Punkte des Eingabepolygons haben die Koordinaten: $p_1 = (6,8)$, $p_2 = (7,11)$, $p_3 = (8,8)$, $p_4 = (11,12)$, $p_5 = (9,5)$.



(a) Zu clippendes Polygon



(b) Polygon-Szene von oben

Abbildung 1: Polygon und BSP

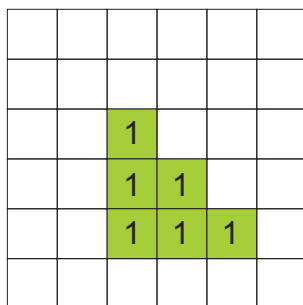
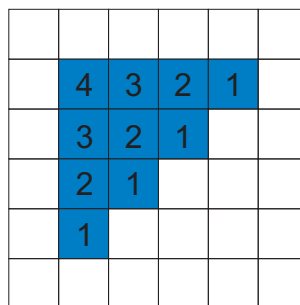
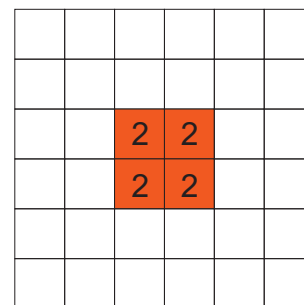
Aufgabe 4.2 (BSP-Bäume)**[4 Punkte]**

Innerhalb der Abbildung 1(b) ist eine Szene bestehend aus rechteckigen Polygonen, die senkrecht über einer gemeinsamen Ebene stehen, von oben dargestellt. Zusätzlich ist der Augenpunkt angegeben, von dem aus die Szene betrachtet wird.

- Zeichnen Sie den BSP-Baum, der das Polygon A als Wurzel hat und alle Polygone beinhaltet (siehe Skript Seite 346ff. (Kapitel D.5.2.2)). Die Knoten sollen in alphabetischer Reihenfolge eingefügt werden.
- Geben Sie für den BSP-Baum und den in Abbildung 1(b) gegebenen Augenpunkt die Reihenfolge der Polygone an, in der diese ausgegeben werden.

Aufgabe 4.3 (z-Buffer-Verfahren)**[3 Punkte]**

Gegeben seien drei Polygone A , B und C innerhalb einer Szene, die übereinander auf die Bildebene projiziert und verrastert werden sollen (s. Abbildung 2). Die Zahlenwerte repräsentieren dabei die z -Werte. Berechnen Sie die Sichtbarkeit der Polygone mithilfe des z -Buffer-Algorithmus (siehe Skript Seite 354ff. (Kapitel D.5.3.1)), wenn die Polygone in der Reihenfolge A, B, C in die Szene eingefügt werden. Geben Sie hierzu die Inhalte des z -Buffers und des Bild-Buffers wie in der Abbildung 2 nach dem Zeichnen des zweiten und dritten Polygons an.

(a) Polygon A (b) Polygon B (c) Polygon C Abbildung 2: Polygone A , B und C mit den zugehörigen z -Werten innerhalb der Szene.**Aufgabe 4.4 (Polygone in Java)****[5 Punkte]**

Laden Sie den Coderahmen von der MMI-Übungsseite zu dieser Aufgabe herunter. Ziel dieser Aufgabe ist es, durch zweimaliges Klicken mit der Maus zwei Punkte auf dem in Abbildung 3 angezeigten Panel mit dem Polygon festzulegen. Diese beiden Punkte definieren damit den Start- und Endpunkt einer Strecke s (s. Abbildung 4). Zu dieser Strecke soll dann die Länge der Teilstrecken s_p ausgegeben werden, die innerhalb des Polygons verlaufen. Diese ist im Koordinatensystem des Polygons anzugeben.

- Implementieren Sie in der Klasse `PolygonPanel` die Methode `private Point2D.Double intersectLineLine(Point2D.Double a1, Point2D.Double b1, Point2D.Double a2, Point2D.Double b2)`, welche die Strecken

$$g : \mathbf{x} = \mathbf{a}_1 + (\mathbf{b}_1 - \mathbf{a}_1)t \quad \text{für } 0 \leq t \leq 1$$

$$h : \mathbf{x} = \mathbf{a}_2 + (\mathbf{b}_2 - \mathbf{a}_2)s \quad \text{für } 0 \leq s \leq 1$$

untersucht und die Parameterwerte s und t des Schnittpunktes der zugehörigen Geraden zurückgibt. Sind diese parallel, so ist $+\infty$ oder $-\infty$ ein sinnvolles Ergebnis.

- d) Implementieren Sie in der Klasse `PolygonPanel` die Methode `private boolean isInsidePolygon(Point2D.Double p)`, die überprüft, ob sich der Punkt `p` innerhalb des Polygons befindet. In diesem Fall gibt die Methode `true` zurück, ansonsten `false`. Das Polygon ist dabei eine Membervariable der Klasse. Bitte verwenden Sie keine bestehenden Bibliotheken für den Punkt-in-Polygon-Test.

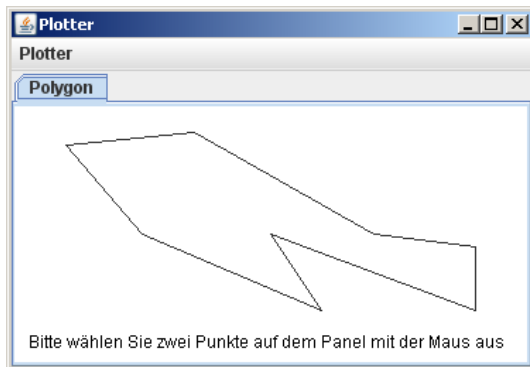


Abbildung 3: Ansicht vor der Selektion

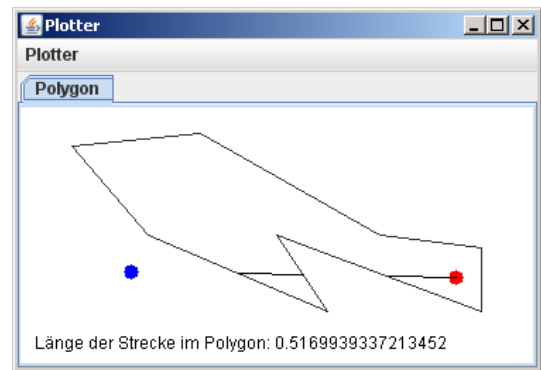


Abbildung 4: Markierte Strecke nach der Auswahl